

ALGORITMA DAN STRUKTUR DATA(ASD)
TUGAS PRAKTIKUM 7 ALGORITMA PENGULANGAN



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TUGAS PRAKTIKUM ASD – ALGORITMA PENGURUTAN

- A. Implementasikan 6 algoritma sorting dalam bentuk ascending dan descending. Tuliskan source code, hasil running program dan analisis. Berikan juga kesimpulan hasil praktikum.

Data Array acak yang digunakan: (58, 78, 97, 48, 67, 86, 6, 57, 76, 95)

1. Bubble Sort

Kode:

```
1. #include <stdio.h>
2.
3. void swap(int *a, int *b);
4. void printarray(int arr[], int n);
5. void bubblesort(int arr[], int n, int asc);
6.
7. int main() {
8.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
9.     int n = sizeof(data) / sizeof(data[0]);
10.    int arr[10];
11.    int i;
12.
13.    printf("array awal: ");
14.    printarray(data, n);
15.
16.    for(i = 0; i < n; i++) arr[i] = data[i];
17.    bubblesort(arr, n, 1);
18.    printf("bubble sort (asc) : ");
19.    printarray(arr, n);
20.
21.    for(i = 0; i < n; i++) arr[i] = data[i];
22.    bubblesort(arr, n, 0);
23.    printf("bubble sort (desc): ");
24.    printarray(arr, n);
25.
26.    return 0;
27. }
28.
29. void swap(int *a, int *b) {
30.     int temp = *a;
31.     *a = *b;
32.     *b = temp;
33. }
34.
35. void printarray(int arr[], int n) {
36.     int i;
```

```

37.     for(i = 0; i < n; i++) printf("%d ", arr[i]);
38.     printf("\n");
39. }
40.
41. void bubblesort(int arr[], int n, int asc) {
42.     int i, j, swapped;
43.     for(i = 0; i < n - 1; i++) {
44.         swapped = 0;
45.         for(j = 0; j < n - i - 1; j++) {
46.             if(asc ? (arr[j] > arr[j + 1]) : (arr[j] <
arr[j + 1])) {
47.                 swap(&arr[j], &arr[j + 1]);
48.                 swapped = 1;
49.             }
50.         }
51.         if(!swapped) break;
52.     }
53. }

```

Output:

```

array awal: 58 78 97 48 67 86 6 57 76 95
bubble sort (asc) : 6 48 57 58 67 76 78 86 95 97
bubble sort (desc): 97 95 86 78 76 67 58 57 48 6

```

Analisis Program:

Analisis untuk program Bubble Sort menunjukkan bahwa kode ini bekerja dengan membaca array dari indeks pertama sampai terakhir untuk membandingkan dua nilai yang bersebelahan. Jika instruksinya adalah menaik, dan nilai sebelah kiri lebih besar dari nilai sebelah kanan, program akan menukar posisi keduanya. Proses perbandingan ini diulang dari awal secara terus-menerus sampai program tidak menemukan ada lagi nilai yang perlu ditukar posisinya. Kelebihan dari algoritma ini adalah logika kodenya yang sangat pendek dan mudah ditulis. Namun, kekurangannya adalah proses kerjanya memakan waktu paling lambat karena harus memeriksa dan mengulang seluruh pengecekan data satu per satu, bahkan jika hanya ada satu angka yang posisinya salah.

2. Selection Sort

Kode:

```

1. #include <stdio.h>
2.
3. void swap(int *a, int *b);
4. void printarray(int arr[], int n);
5. void selectionsort(int arr[], int n, int asc);
6.
7. int main() {

```

```

8.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
9.     int n = sizeof(data) / sizeof(data[0]);
10.    int arr[10];
11.    int i;
12.
13.    printf("array awal: ");
14.    printarray(data, n);
15.
16.    for(i = 0; i < n; i++) arr[i] = data[i];
17.    selectionsort(arr, n, 1);
18.    printf("selection sort (asc) : ");
19.    printarray(arr, n);
20.
21.    for(i = 0; i < n; i++) arr[i] = data[i];
22.    selectionsort(arr, n, 0);
23.    printf("selection sort (desc): ");
24.    printarray(arr, n);
25.
26.    return 0;
27. }
28.
29. void swap(int *a, int *b) {
30.     int temp = *a;
31.     *a = *b;
32.     *b = temp;
33. }
34.
35. void printarray(int arr[], int n) {
36.     int i;
37.     for(i = 0; i < n; i++) printf("%d ", arr[i]);
38.     printf("\n");
39. }
40.
41. void selectionsort(int arr[], int n, int asc) {
42.     int i, j, extremeidx;
43.     for(i = 0; i < n - 1; i++) {
44.         extremeidx = i;
45.         for(j = i + 1; j < n; j++) {
46.             if(asc ? (arr[j] < arr[extremeidx]) : (arr[j]
> arr[extremeidx])) {
47.                 extremeidx = j;
48.             }
49.         }

```

```

50.         swap(&arr[extremeidx], &arr[i]);
51.     }
52. }

```

Output:

```

array awal: 58 78 97 48 67 86 6 57 76 95
selection sort (asc) : 6 48 57 58 67 76 78 86 95 97
selection sort (desc): 97 95 86 78 76 67 58 57 48 6

```

Analisis Program:

Pada program Selection Sort, kodenya bekerja dengan membaca seluruh isi array untuk mencari nilai yang paling kecil. Setelah nilai paling kecil ditemukan, program menukar nilai tersebut dengan nilai yang berada di urutan paling pertama. Kemudian, program mengabaikan urutan pertama tadi, dan mencari lagi nilai paling kecil dari sisa data yang ada untuk ditukar ke urutan kedua, dan proses pemindaian ini diulang sampai urutan terakhir. Algoritma ini memiliki kelebihan berupa jumlah proses pertukaran posisi data yang sangat sedikit jika dibandingkan dengan Bubble Sort. Akan tetapi, kekurangannya adalah program tetap berjalan lambat jika jumlah datanya banyak karena wajib membaca seluruh barisan angka yang tersisa di setiap perulangannya.

3. Insertion Sort

Kode:

```

1. #include <stdio.h>
2.
3. void printarray(int arr[], int n);
4. void insertionsort(int arr[], int n, int asc);
5.
6. int main() {
7.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
8.     int n = sizeof(data) / sizeof(data[0]);
9.     int arr[10];
10.    int i;
11.
12.    printf("array awal: ");
13.    printarray(data, n);
14.
15.    for(i = 0; i < n; i++) arr[i] = data[i];
16.    insertionsort(arr, n, 1);
17.    printf("insertion sort (asc) : ");
18.    printarray(arr, n);
19.
20.    for(i = 0; i < n; i++) arr[i] = data[i];
21.    insertionsort(arr, n, 0);
22.    printf("insertion sort (desc): ");

```

```

23.     printarray(arr, n);
24.
25.     return 0;
26. }
27.
28. void printarray(int arr[], int n) {
29.     int i;
30.     for(i = 0; i < n; i++) printf("%d ", arr[i]);
31.     printf("\n");
32. }
33.
34. void insertionsort(int arr[], int n, int asc) {
35.     int i, j, key;
36.     for(i = 1; i < n; i++) {
37.         key = arr[i];
38.         j = i - 1;
39.         while(j >= 0 && (asc ? (arr[j] > key) : (arr[j] <
key))) {
40.             arr[j + 1] = arr[j];
41.             j = j - 1;
42.         }
43.         arr[j + 1] = key;
44.     }
45. }

```

Output:

```

array awal: 58 78 97 48 67 86 6 57 76 95
insertion sort (asc) : 6 48 57 58 67 76 78 86 95 97
insertion sort (desc): 97 95 86 78 76 67 58 57 48 6

```

Analisis Program:

Untuk program Insertion Sort, program membaca nilai dari kiri ke kanan, dan saat membaca sebuah nilai, ia membandingkannya dengan nilai-nilai yang sudah ada di sebelah kirinya. Jika nilai di kiri lebih besar, program akan menggeser nilai di kiri tersebut ke arah kanan, agar nilai yang sedang dibaca tadi bisa disisipkan ke posisi yang benar di sebelah kiri. Kelebihan dari cara kerja ini adalah prosesnya berjalan cepat jika sebagian besar barisan angkanya sudah dalam keadaan terurut. Sedangkan kekurangannya adalah kurang cepat untuk memproses jumlah data yang sangat banyak karena program harus memindahkan banyak angka ke kanan setiap kali menemukan nilai yang kecil.

4. Shell Sort

Kode:

```
1. #include <stdio.h>
2.
3. void printarray(int arr[], int n);
4. void shellsort(int arr[], int n, int asc);
5.
6. int main() {
7.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
8.     int n = sizeof(data) / sizeof(data[0]);
9.     int arr[10];
10.    int i;
11.
12.    printf("array awal: ");
13.    printarray(data, n);
14.
15.    for(i = 0; i < n; i++) arr[i] = data[i];
16.    shellsort(arr, n, 1);
17.    printf("shell sort (asc) : ");
18.    printarray(arr, n);
19.
20.    for(i = 0; i < n; i++) arr[i] = data[i];
21.    shellsort(arr, n, 0);
22.    printf("shell sort (desc): ");
23.    printarray(arr, n);
24.
25.    return 0;
26. }
27.
28. void printarray(int arr[], int n) {
29.     int i;
30.     for(i = 0; i < n; i++) printf("%d ", arr[i]);
31.     printf("\n");
32. }
33.
34. void shellsort(int arr[], int n, int asc) {
35.     int gap, i, j, temp;
36.     for(gap = n / 2; gap > 0; gap /= 2) {
37.         for(i = gap; i < n; i += 1) {
38.             temp = arr[i];
39.             for(j = i; j >= gap && (asc ? (arr[j - gap] >
temp) : (arr[j - gap] < temp)); j -= gap) {
40.                 arr[j] = arr[j - gap];
```

```

41.         }
42.         arr[j] = temp;
43.     }
44. }
45. }

```

Output:

```

array awal: 58 78 97 48 67 86 6 57 76 95
shell sort (asc) : 6 48 57 58 67 76 78 86 95 97
shell sort (desc): 97 95 86 78 76 67 58 57 48 6

```

Analisis Program:

Program Shell Sort merupakan versi pengembangan dari Insertion Sort, di mana program tidak langsung membandingkan angka yang bersebelahan. Program menggunakan sebuah jarak pembagian tertentu, misalnya membandingkan urutan pertama dengan urutan kelima, lalu menukarnya jika perlu. Setelah semua nilai dengan jarak tertentu selesai diurutkan, jarak tersebut diperkecil, lalu program membandingkan lagi sampai akhirnya jaraknya menjadi berdekatan secara normal. Kelebihannya adalah program mampu memindahkan angka kecil yang berada di ujung paling kanan langsung ke posisi kiri dengan cepat tanpa harus digeser satu per satu. Kekurangan utamanya adalah tingkat kerumitan yang tinggi untuk dipahami logika matematisnya saat menentukan jarak pembagian tersebut.

5. Merge Sort

Kode:

```

1. #include <stdio.h>
2.
3. void printarray(int arr[], int n);
4. void merge(int arr[], int l, int m, int r, int asc);
5. void mergesort(int arr[], int l, int r, int asc);
6.
7. int main() {
8.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
9.     int n = sizeof(data) / sizeof(data[0]);
10.    int arr[10];
11.    int i;
12.
13.    printf("array awal: ");
14.    printarray(data, n);
15.
16.    for(i = 0; i < n; i++) arr[i] = data[i];
17.    mergesort(arr, 0, n - 1, 1);
18.    printf("merge sort (asc) : ");
19.    printarray(arr, n);
20.

```



```

21.     for(i = 0; i < n; i++) arr[i] = data[i];
22.     mergesort(arr, 0, n - 1, 0);
23.     printf("merge sort (desc): ");
24.     printarray(arr, n);
25.
26.     return 0;
27. }
28.
29. void printarray(int arr[], int n) {
30.     int i;
31.     for(i = 0; i < n; i++) printf("%d ", arr[i]);
32.     printf("\n");
33. }
34.
35. void merge(int arr[], int l, int m, int r, int asc) {
36.     int i, j, k;
37.     int n1 = m - l + 1;
38.     int n2 = r - m;
39.     int arr1[n1], arr2[n2];
40.
41.     for(i = 0; i < n1; i++) arr1[i] = arr[l + i];
42.     for(j = 0; j < n2; j++) arr2[j] = arr[m + 1 + j];
43.
44.     i = 0;
45.     j = 0;
46.     k = l;
47.     while(i < n1 && j < n2) {
48.         if(asc ? (arr1[i] <= arr2[j]) : (arr1[i] >=
arr2[j])) {
49.             arr[k++] = arr1[i++];
50.         } else {
51.             arr[k++] = arr2[j++];
52.         }
53.     }
54.     while(i < n1) arr[k++] = arr1[i++];
55.     while(j < n2) arr[k++] = arr2[j++];
56. }
57.
58. void mergesort(int arr[], int l, int r, int asc) {
59.     if(l < r) {
60.         int m = l + (r - l) / 2;
61.         mergesort(arr, l, m, asc);
62.         mergesort(arr, m + 1, r, asc);

```

```

63.         merge(arr, l, m, r, asc);
64.     }
65. }

```

Output:

```

array awal: 58 78 97 48 67 86 6 57 76 95
merge sort (asc) : 6 48 57 58 67 76 78 86 95 97
merge sort (desc): 97 95 86 78 76 67 58 57 48 6

```

Analisis Program:

Pada analisis program Merge Sort, program ini memotong barisan nilai menjadi dua bagian secara terus-menerus sampai semua nilai terpisah menjadi array yang masing-masing hanya berisi satu angka tunggal. Setelah semuanya terpisah, program mulai menggabungkan kembali array tersebut satu per satu, dan saat menggabungkan dua array, program sekaligus membandingkan isinya agar hasilnya langsung terurut. Kelebihan algoritma ini adalah waktu kerjanya yang selalu stabil dan sangat cepat untuk mengurutkan jumlah angka yang sangat banyak, baik datanya sangat acak maupun sudah rapi. Kekurangannya adalah program ini membutuhkan ketersediaan memori komputer tambahan karena harus membuat array baru sebagai tempat menampung nilai saat proses penggabungan berlangsung.

6. Quick Sort

Kode:

```

1. #include <stdio.h>
2.
3. void swap(int *a, int *b);
4. void printarray(int arr[], int n);
5. int partition(int arr[], int low, int high, int asc);
6. void quicksort(int arr[], int low, int high, int asc);
7.
8. int main() {
9.     int data[] = {58, 78, 97, 48, 67, 86, 6, 57, 76, 95};
10.    int n = sizeof(data) / sizeof(data[0]);
11.    int arr[10];
12.    int i;
13.
14.    printf("array awal: ");
15.    printarray(data, n);
16.
17.    for(i = 0; i < n; i++) arr[i] = data[i];
18.    quicksort(arr, 0, n - 1, 1);
19.    printf("quick sort (asc) : ");
20.    printarray(arr, n);
21.
22.    for(i = 0; i < n; i++) arr[i] = data[i];

```

```

23.     quicksort(arr, 0, n - 1, 0);
24.     printf("quick sort (desc): ");
25.     printarray(arr, n);
26.
27.     return 0;
28. }
29.
30. void swap(int *a, int *b) {
31.     int temp = *a;
32.     *a = *b;
33.     *b = temp;
34. }
35.
36. void printarray(int arr[], int n) {
37.     int i;
38.     for(i = 0; i < n; i++) printf("%d ", arr[i]);
39.     printf("\n");
40. }
41.
42. int partition(int arr[], int low, int high, int asc) {
43.     int pivot = arr[high];
44.     int i = (low - 1);
45.     int j;
46.
47.     for(j = low; j <= high - 1; j++) {
48.         if(asc ? (arr[j] < pivot) : (arr[j] > pivot)) {
49.             i++;
50.             swap(&arr[i], &arr[j]);
51.         }
52.     }
53.     swap(&arr[i + 1], &arr[high]);
54.     return (i + 1);
55. }
56.
57. void quicksort(int arr[], int low, int high, int asc) {
58.     if(low < high) {
59.         int pi = partition(arr, low, high, asc);
60.         quicksort(arr, low, pi - 1, asc);
61.         quicksort(arr, pi + 1, high, asc);
62.     }
63. }

```

Output:

```
array awal: 58 78 97 48 67 86 6 57 76 95  
quick sort (asc) : 6 48 57 58 67 76 78 86 95 97  
quick sort (desc): 97 95 86 78 76 67 58 57 48 6
```

Analisis Program:

Terakhir, untuk program Quick Sort, kodenya bekerja dengan memilih satu nilai dari dalam data untuk dijadikan titik penentu. Program kemudian memisahkan angka-angka lainnya dengan memindah nilai yang lebih kecil dari titik penentu ke sebelah kiri, dan nilai yang lebih besar ke sebelah kanan. Setelah terpisah, program memproses ulang kelompok kiri dan kanan tersebut sampai tidak ada lagi yang bisa dipisahkan. Kelebihan Quick Sort adalah eksekusinya yang sangat cepat untuk memproses data besar dan tidak memakan banyak memori tambahan karena pertukaran datanya dilakukan secara langsung di tempat yang sama. Kekurangannya adalah proses bisa menjadi sangat lambat apabila program terus-menerus memilih nilai terbesar atau terkecil sebagai titik penentu.

Kesimpulan Hasil Praktikum

Berdasarkan analisis keenam program tersebut, kesimpulan dari praktikum ini membuktikan bahwa untuk menyelesaikan masalah pengurutan kumpulan data, komputer memiliki banyak opsi metode yang berbeda dengan pendekatan perulangan dan perbandingan yang berbeda pula. Algoritma dasar seperti Bubble, Selection, dan Insertion memiliki performa yang kurang baik untuk mengolah jumlah data yang banyak karena menggunakan perulangan bertingkat secara langsung, sementara algoritma Shell, Merge, dan Quick jauh lebih cepat karena menggunakan cara pemisahan lompatan data.

Terkait penggunaan memori, mayoritas algoritma menghemat kapasitas penyimpanan komputer karena memindahkan nilai langsung pada ruang data asalnya, kecuali Merge Sort yang secara khusus membutuhkan penciptaan array sementara di dalam sistem. Selain itu, hasil praktikum juga menunjukkan bahwa pengubahan arah urutan nilai menjadi menaik atau menurun sangat sederhana untuk dilakukan. Penyesuaian ini tidak merombak kerangka kerja sistem, karena program hanya membutuhkan penggantian tanda perbandingan matematis tanpa mengubah susunan kode secara keseluruhan.